

Recursion, Computability and Algorithms Notes

An exploration of the beautiful topic of recursive functions, computability, algorithms
and The Incompleteness Theorems

Eklavya Raman

June 29, 2025

Contents

I	Theory of Recursive Functions and Effective Computability	4
1	Recursive Functions	5
1.1	Algorithms	5
1.1.1	What is an algorithm?	5
1.1.2	What are the boundations of an Algorithm?	5
1.2	Primitive Recursive Functions	6
1.2.1	What are recursive functions?	6
1.2.2	Diagonalization	7
1.2.3	Fun Interlude	8
1.2.4	Turing Characterization	9
1.2.5	Kleene Characterization	10
1.3	The Basic Result	11
1.3.1	Partial Recursive Functions	11
1.3.2	Church's Thesis	12
1.4	Godel Numbers, Universality, s-m-n Theorem	12
1.4.1	Godel Numbers	12
1.4.2	Universality	13
1.4.3	s-m-n Theorem	13
1.5	The Halting Problem	14
1.5.1	Recursive Unsolvability	14
1.6	Recursiveness	15
1.6.1	Codings	15
1.6.2	The mu (μ) operator	16
2	Unsolvable Problems	18
2.1	Further Examples of Recursive Unsolvability	18
2.1.1	Methods of Proving Unsolvability	18
2.2	Existence of certain partial recursive functions	19
2.2.1	Extension of Partial Recursive Functions	19
2.3	Exercises	20
2.3.1	Exercies from Chapter 1	20
II	Data Structures and Algorithms - Analysis and Design	23
3	Basic Terminology and Definitions	24
3.1	Foundational Concepts	24
3.1.1	Introduction	24
3.1.2	Definitions	24
4	Basic Concepts of Algorithm Analysis and Design	26
4.1	Insertion Sort	26
4.1.1	The Sorting Problem	26
III	Extra Reading - Recursion and Incompleteness Theorems	27
5	Introduction and Definitions (Incompleteness)	28

5.1	Statement of the Problem and Prerequisite Definitions	28
5.1.1	Definitions	28
5.2	Requirements for a deductive system	29

List of Theorems

1.1	Theorem (The Basic Result - 1)	11
1.2	Theorem (The Basic Result - 2)	11
1.3	Theorem (The Basic Result - 3)	11
1.4	Theorem (Church's Thesis)	12
1.5	Theorem (Countability of Recursive Functions)	12
1.6	Theorem (Existence of Non-Recursive Functions)	12
1.7	Theorem (Countability of Indices)	12
1.8	Theorem (Enumeration Theorem)	13
1.9	Theorem (s-m-n Theorem)	13
1.10	Theorem (Composite of Partial Recursive Functions are Partial Recursive)	14
1.11	Theorem (Recursive Unsolvability of the Halting Problem)	14
1.12	Theorem (The Halting Problem for Total Functions)	15
1.13	Theorem (Correspondence of partial functions with algorithmic mapping on uncoded expressions)	16
1.14	Theorem (The Mu Theorem)	16
1.15	Theorem (Kleene Normal-form Theorem)	17
1.16	Theorem (Ex. 2-8 theorem)	17
2.1	Theorem (Unsolvable (a))	18
2.2	Theorem (Extension of Partial Recursive Functions)	19
2.3	Theorem (Existence of Partial Recursive Functions not obtained by μ)	19
2.4	Theorem (Important Corollary of the Halting Problem)	19
5.1	Theorem (The Incompleteness Theorem)	28

Part I

Theory of Recursive Functions and Effective Computability

Chapter 1

Recursive Functions

1.1 Algorithms

1.1.1 What is an algorithm?

Algorithms are sets of instructions of a given finite size. We need computing agents, storage facilities for executing the algorithms and managing, storing the inputs and instructions. The instructions are carried out in a stepwise and deterministic way by the computing agent which is usually a human.

Definition 1.1.1.1

Deterministic Process

A deterministic process is one which does not rely on random methods, i.e. dice. In other words it follows a set of known and predictable steps.

Definition 1.1.1.2

P and LP Symbolism

Here we define both in terms of sets of symbolic expressions and their meanings, in a sense.

- P symbolisms are the specific sets of symbolic expressions accepted as instructions, inputs, or outputs.
- How the above instructions and inputs determine how we can compute and identify outputs is known as LP Symbolism.

1.1.2 What are the foundations of an Algorithm?

Lets begin by simple statements -

1. Theoretically, there does not need to be a fixed finite bound on the size of inputs, that is, we can have an arbitrarily large set of inputs or inputs that are arbitrarily large.
2. Theoretically, there does not need to be a fixed finite bound on the size of the set of instructions, that is, we can have an arbitrarily large number of instructions.
3. Therefore it follows that, theoretically, there does not need to be a fixed finite bound on the size of memory or storage available.

However, in a practical scenario, we encounter two types of functions -

1. **Functions computable by finite-state machine** - These can be computed using a finite state machine (a machine with finite memory capacity), no matter the size of the inputs.
For example - $\lambda x[2x]$ as x increases.
2. The converse of the above.
For example - $\lambda x[x^2]$ as x increases.

We must also note the following -

- There is to be a fixed finite bound on the capacity of the ability of the computing agent (L).

Note 1.1.2.1

Limiting the Computing Agent (L)

With a sufficiently flexible P-symbolism, meaning that we can modify the P-symbolism such that excessive computing capacities of L are converted to demands on the memory capacity, we can limit L. The ways are enlisted below -

1. Limit L to simple clerical operations.
2. A temporary memory of fixed size which preserves the 'place' we have reached within the set of instructions or inputs.
3. A set of rules to determine the next state of the temporary memory and the next clerical operation to be performed.

This becomes clearer when we consider *Turing Machines* later in the notes.

Lastly, we require that computation ends after some finite number of steps.

1.2 Primitive Recursive Functions

1.2.1 What are recursive functions?

Definition 1.2.1.1

Recursive Functions

Functions obtainable by a recursive definition, that is, values of the functions for given arguments are directly related to values of the same functions for simpler arguments (most usually a set of base constant values). Recursive functions can often be made to serve as algorithms.

The finest and most beautiful example of a recursive function is the ***Fibonacci Sequence***.

Example 1.2.1.1 -

$f : \mathbb{N} \rightarrow \mathbb{N}$ given by -

$$\begin{aligned} f(0) &= 1 \\ f(1) &= 1 \\ f(x+2) &= f(x+1) + f(x) \end{aligned}$$

The algebraic definition of the fibonacci sequence is -

$$f(x) = \frac{\sqrt{5}}{5} \left(\frac{1+\sqrt{5}}{2} \right)^{x+1} - \frac{\sqrt{5}}{5} \left(\frac{1-\sqrt{5}}{2} \right)^{x+1}$$

Definition 1.2.1.2

Class of Primitive Recursive Functions

Denoting by $\mathcal{C}$, the class includes -

1. All constant functions.
2. The successor function.
3. All identity functions.
4. If f is a function of k variables in $\mathcal{C}$ and $g_1, g_2, \dots, g_k$ are each functions of m variables in $\mathcal{C}$,

then the function -

$$\lambda x_1, x_2 \dots x_m [f(g_1(x_1, \dots, x_m), \dots, g_k(x_1, \dots, x_m))]$$

is also in $\mathcal{C}$.

5. If h is a function of $k+1$ variables in $\mathcal{C}$ and g is a function of $k-1$ variables in $\mathcal{C}$, then the unique function f of k variables satisfying -

$$f(0, x_2, \dots, x_k) = g(x_2, \dots, x_k)$$

$$f(y+1, x_2, \dots, x_k) = h(y, f(y, x_2, \dots, x_k), x_2, \dots, x_k)$$

is also in $\mathcal{C}$

Note 1.2.1.1

The above definition leads to the fact that -

For any f , f is primitive recursive $\iff$ there is a finite sequence of functions $f_1, f_2, \dots, f_n$ such that $f_n = f$ and for each $j \leq n$, f_j is in $\mathcal{C}$ or is directly obtainable from some of the f_i , $i \leq j$ using 4. or 5. Almost all algorithmic functions in ordinary mathematics can be shown to be primitive recursive.

Definition 1.2.1.3

Derivation

If a sequence as mentioned above is described with a specification of how each function in the sequence is obtained, we say that we have a derivation for f as a primitive recursive function.

Note 1.2.1.2

For more on the usefulness of primitive recursiveness, Peter [1951, pp. 1-67]

Even if primitive recursive functions are quite accurate, they are not completely generalized and do not work for all functions computable by algorithm. Example is the *Ackermann Generalized Exponential* function. Hence these are not accurate formal counterpart to the notion of algorithmic function. We can also provide functions that we can prove have an algorithmic derivation but we do not know yet how to find the derivation.

Example 1.2.1.2 -

Goldbach's Conjecture - The conjecture states that every even number greater than 2 is the sum of 2 primes. We define a function h by -

$$h(x) = \begin{cases} 1 & \text{if conjecture true;} \\ 0 & \text{otherwise} \end{cases}$$

It is enough to see that h is a constant function to conclude that it is indeed primitive recursive. However, we do not know how to identify the correct derivation.

1.2.2 Diagonalization

This is similar to the Diagonalization argument by Cantor. It is a method applied to formal classes of algorithmic functions which produces a function outside of the formal class. The diagonalization argument suggests that we can not have a P-Symbolism and LP-Symbolism strong enough to represent all algorithmic functions.

Note 1.2.2.1

Diagonalization presents an interesting possible relation to Godel's Incompleteness theorems.

Formal Characterization

Note 1.2.2.2

We can avoid Diagonalization by allowing sets of instructions for partial functions as well as for total functions. Basically, we allow procedures which don't always terminate and yield an output, i.e. partial functions.

The approach of introducing a class of partial functions was taken first by Kleene, Church, and Turing. The common features of the characterizations of these different researchers were -

1. Formal counterparts to the notion of an Algorithm through P-symbolisms.
2. Formal counterparts for the notion of a partial function computable by algorithm.

1.2.3 Fun Interlude

Code 1.2.3.1

Listing 1.1: Brute Force Algorithm to verify Goldbach's Conjecture

```
import itertools
def is_prime(n):
    if n == 1:
        return False
    elif n == 2:
        return True
    elif n == 3:
        return True
    elif n == 4:
        return False
    else:
        for i in range(2, int(n//2)):
            if n % i == 0:
                return False
        return True

def is_even_geq_2(n):
    if n <= 2:
        return False
    elif n%2!=0:
        return False
    else:
        return True

def verify(n):
    primes = []
    for i in range(2, n + 1):
        if is_prime(i):
            primes.append(i)
    combinations = list(itertools.product(primes, repeat=2))
    for i in combinations:
        a = 0
        for j in i:
            a += j
        if a == n:
            return True
    else:
```

```

        a = 0
    return False

def soln(n):
    for i in range(1, n + 1):
        if is_even_geq_2(i):
            if verify(i):
                pass
            else:
                return False
    return True

```

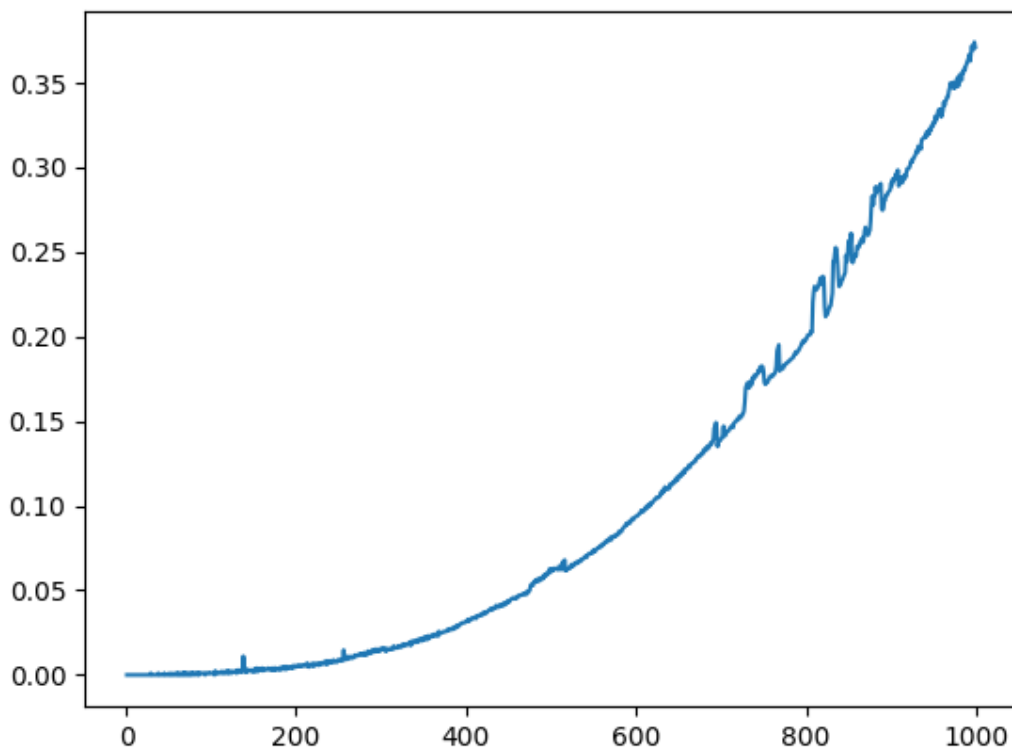


Figure 1.1: Runtime in secs for the first 1000 integers

We can notice some interesting peaks and valleys in an otherwise smooth graph. To be looked at later.

1.2.4 Turing Characterization

We consider first a physical picture of the Turing Characterization for the purpose of illustration.

Let us say we have a machine that can perform 4 operations - A, B, C, D. It performs 1 operation per unit time. The device can take on one of a fixed finite set of possible mechanical configurations called the **Internal State**. The device must follow a set of deterministic rules.

Let there be 2 possible, that is binary conditions for the cell being examined by the machine. Then the set of rules determining the behavior of the device may be expressed as *quadruples* that include the internal state, the condition of the cell, an operation and a further internal state. To ensure consistency, we require that for each internal state and cell condition, there is exactly one quadruple.

Definition 1.2.4.1**Turing Machine**

A Turing Machine is a device that can perform a set of operations on a set of inputs, and can be described by a set of quadruples as described above, provided that the consistency condition is satisfied, and the set of quadruples is constructed using a finite number of internal states.

A more rigorous definition is given as follows -

Let $T = 0, 1$ and $S = 0, 1, 2, 3$. Then a Turing Machine can be defined as a mapping from a finite subset of $\mathbb{N} \times T$ into $S \times \mathbb{N}$

Note 1.2.4.1

The Turing Machine is a physical model of computation, and is not a formal model. It is a model of a computing agent that can perform operations on inputs and outputs in a deterministic manner.

Definition 1.2.4.2**Turing Computable Function**

A function is Turing computable if there exists a Turing Machine that can compute the function.

Example 1.2.4.3 -

We verify that the function $\lambda x[2x]$ is yielded by the following Turing Machine -

$$q_0 1 B q_1 \quad (1.2.4.1)$$

$$q_1 B R q_2 \quad (1.2.4.2)$$

$$q_2 1 B q_3 \quad (1.2.4.3)$$

$$q_3 B R q_4 \quad (1.2.4.4)$$

$$q_4 1 R q_4 \quad (1.2.4.5)$$

$$q_4 B R q_5 \quad (1.2.4.6)$$

$$q_5 1 R q_5 \quad (1.2.4.7)$$

$$q_5 B 1 q_6 \quad (1.2.4.8)$$

$$q_6 1 R q_6 \quad (1.2.4.9)$$

$$q_6 B 1 q_7 \quad (1.2.4.10)$$

$$q_7 1 L q_7 \quad (1.2.4.11)$$

$$q_7 B L q_8 \quad (1.2.4.12)$$

$$q_8 1 L q_1 \quad (1.2.4.13)$$

$$q_1 1 L q_1 \quad (1.2.4.14)$$

The machine takes a tape with $x+1$ consecutive 1s and outputs $2x+1$ 1s. q_0 is the leftmost cell containing 1. Let the input string be -

$$\dots B 1 1 1 B \dots$$

Following the above quadruples, we can see that the machine will output -

$$\dots B 1 1 1 1 1 B \dots$$

Turing Machines, like other characterizations, give a class of algorithmic partial functions. The class may seem to be limited, but it provides a meaningful P-symbolism and LP specifications.

1.2.5 Kleene Characterization

Kleene's characterization is very much similar to the kind we used in example 1.2.1.1 to describe the Fibonacci sequence. The Kleene characterization is a formalization of the idea of a recursive function. It is based on the idea of a recursive definition, where the value of a function for a given argument is defined in terms of the values of the same function for simpler arguments. The P symbolism

is the same as the one used to describe functions, equations, and variables in general mathematics, *except that for describing a term such as $x + 2$, we must write $x + 1 + 1$.*

Note 1.2.5.1

Every primitive recursive function can be expressed in Kleene's notation or the P-symbolism of the Kleene Characterization. As such we can find a **derivation** for every primitive recursive function in Kleene's notation.

Limitations of Kleene's Characterization

Kleene's Characterization does not guarantee that the computation is uniquely determined by the input or that one input may yield multiple outputs. It also does not guarantee that an output will be generated for every input.

To avoid such limitations, we do the following -

Given any P, we generate a list of all equations deducible from P. Then for the given P and any input, we define the **principal output** as the first output that can be deduced from the list of equations. We only allow one principal output for each input.

Note 1.2.5.2

Note that the above characterization also forms a class of algorithmic partial functions, similar to the Turing Characterization.

1.3 The Basic Result

The following is a fundamental result, which will be used throughout the rest of the notes. It is a result that shows the equivalence of the different characterizations of algorithmic functions.

1.3.1 Partial Recursive Functions

Theorem 1.1 (The Basic Result - 1). ***The Basic Result - 1***

The proposed characterizations of algorithmic functions by Turing, Kleene, and others are equivalent in the sense that they yield the same class of algorithmic partial functions.

Definition 1.3.1.1

Partial Recursive Functions

A function is said to be a partial recursive function if it is computable by a Turing Machine, or if it is computable by a Kleene Characterization, or if it is computable by any other characterization of algorithmic functions.

That is, it is the class of partial functions (and hence total functions) generated by the formal characterizations as mentioned above.

Theorem 1.2 (The Basic Result - 2). ***The Basic Result - 2***

*A wide variety of partial functions have been shown to be partial recursive functions. **Theorem 1.3.1.1** and this provides evidence that the class of partial recursive functions is inclusive of a large number of functions computable by algorithm.*

Theorem 1.3 (The Basic Result - 3). ***The Basic Result - 3***

Given any set of instructions P from one characterizations, we can find a set of instructions P' from another characterization for the same partial function.

1.3.2 Church's Thesis

Theorem 1.4 (Church's Thesis). ***Church's Thesis***

We accept, on empirical grounds, that the characterizations mentioned above are satisfactory formalizations of the notion of algorithms and algorithmic functions.

Note 1.3.2.1

This book relies on the Church's Thesis, and its assumption, to provide informal but rigorous proofs of the results presented in the book.

The Church's Thesis is not a theorem, but an assumption that is widely accepted in the field of computability. It is based on the empirical evidence that the characterizations mentioned above are sufficient to describe all algorithmic functions.

It should also be noted that there are ways to formally construct the same proofs that have been developed informally. The formal proofs are not presented in this book, but the informal proofs are rigorous and provide a good understanding of the results.

1.4 Godel Numbers, Universality, s-m-n Theorem

1.4.1 Godel Numbers

Assume we can list all the sets of instructions P from one characterization. Such a procedure associates an integer x with the $(x + 1)^{th}$ set of instructions in the list of all sets of instructions.

Definition 1.4.1.1

Godel Number

A Godel Number is an integer that is associated with a set of instructions P from one characterization. It is used to identify the set of instructions in the list of all sets of instructions. It is also sometimes referred to as the *index* of the set of instructions.

Let P_x be the set of instructions associated with the Godel Number x . Then, the partial function computable by the set of instructions P_x is denoted by φ_x^k . Thus, x is also the *index* or **Godel Number** for the partial function associated with P_x .

Theorem 1.5 (Countability of Recursive Functions). *There are exactly $\aleph_0$ Godel Numbers (since there are countably many sets of instructions). Therefore, there are countably many partial recursive functions and countably many recursive functions.*

Proof. The proof is based on the fact that there are countably many sets of instructions, and hence countably many Godel Numbers. Since each Godel Number is associated with a set of instructions, and each set of instructions can be used to compute a partial recursive function, it follows that there are at most countably many partial recursive functions.

Since all constant functions are recursive, there are at least countably many recursive functions. Therefore, there are exactly countably many recursive functions. $\square$

Theorem 1.6 (Existence of Non-Recursive Functions). *There exist functions that are not recursive.*

Proof. We succeeded in showing there are exactly $\aleph_0$ recursive functions. By Cantor's Theorem, there exist $2^{\aleph_0} = \aleph_1$ many functions. Hence, by PHP, there must exist function(s) that are not recursive. $\square$

Theorem 1.7 (Countability of Indices). *Each partial recursive function has $\aleph_0$ indices or Godel Numbers.*

Proof. Let the partial recursive function be φ_{x_0} . Let P_{x_0} be the set of instructions associated with φ_{x_0} . Let $m-1$ denote the highest subscript integer of any internal state in P_{x_0} . Then we can modify P_{x_0} by adding redundant quadruples - $q_m 11 q_m, q_{m+1} 11 q_{m+1}, \dots, q_{m+k} 11 q_{m+k}$ where k is a positive integer. Depending on k , we get the unique set of instructions $P_{x_0}^k$. Since k is a positive integer, we can have countably many sets of instructions for φ_{x_0} . $\square$

1.4.2 Universality

Theorem 1.8 (Enumeration Theorem). **Enumeration Theorem**

There exists a partial recursive function Ψ such that for every partial recursive function φ_x and any y , $\Psi(x, y) = \varphi_x(y)$.

Proof. We define a function Ψ algorithmically as follows -

Given $\langle x, y \rangle$, find P_x (This is itself algorithmic), then apply P_x to y . If and when $\varphi_x(y)$ terminates, take the output as the output of $\Psi(x, y)$. If it does not terminate, then $\Psi(x, y)$ does not terminate.

Therefore, Ψ can be defined as -

$$\Psi(x, y) = \begin{cases} \varphi_x(y) & \text{if } \varphi_x(y) \text{ converges} \\ \text{divergent} & \text{otherwise} \end{cases}$$

By Church's Thesis, since Ψ can be computed algorithmically, Ψ is a partial recursive function. Hence, we can say that $\Psi = \varphi_{x_0}^{(2)}$ for some x_0 , that is a partial recursive function of 2 variables with Godel Number x_0 . $\square$

The given proof can be generalized to show that there exists a partial recursive function Ψ such that for every partial recursive function $\varphi_x^{(n)}$ and $y_1, y_2, \dots, y_n$ -

$$\Psi(x, y_1, y_2, \dots, y_n) = \varphi_x(y_1, y_2, \dots, y_n)$$

where n is a positive integer. The function Ψ is called a **universal partial function**.

Note 1.4.2.1

An alternate way to denote the function is -

$$\Psi(x, y_1, y_2, \dots, y_n) = \varphi_{x_1}^{(n+1)}$$

for some x_1 .

The universal partial function essentially takes as input the Godel Number of a partial recursive function and the inputs to that function, and outputs the result of the function. In other words, it simulates the computation of any partial recursive function given its Godel Number and inputs.

1.4.3 s-m-n Theorem

Theorem 1.9 (s-m-n Theorem). **s-m-n Theorem**

For every $m, n \geq 1$, there exists a recursive function s_n^m of $m+1$ variables such that for all $x, y_1, y_2, \dots, y_m$ -

$$\lambda z_1 z_2 \dots z_n [\varphi_x^{(m+n)}(y_1, y_2, \dots, y_m, z_1, z_2, \dots, z_n)] = \varphi_{s_n^m(x, y_1, y_2, \dots, y_m)}^{(n)}$$

Proof. Consider the family of all partial functions of n variables which are expressible as -

$$\lambda z_1 z_2 \dots z_n [\varphi_x^{(m+n)}(y_1, y_2, \dots, y_m, z_1, z_2, \dots, z_n)]$$

for some x and $y_1, y_2, \dots, y_m$.

Note that this is a new formal characterization offer a class of partial recursive functions of n variables. By **The Basic Result - 3**, there exists a effective procedure for going from sets of instructions in this new characterization to sets of instructions in our standard characterization. Therefore, there must be a recursive function f of n variables such that -

$$\lambda z_1, z_2 \dots z_n [\varphi_x^{(m+n)}(y_1, y_2, \dots, y_m, z_1, z_2, \dots, z_n)] = \varphi_{f(x, y_1, y_2, \dots, y_m)}^{(n)}$$

This function f is the function s_n^m . Here, we have generalized the proof given within the book. $\square$

Note 1.4.3.1

In essence, the s-m-n theorem states that for every partial recursive function of $m + n$ variables, there exists a partial recursive function of n variables that can be obtained by fixing the first m variables. It is achieved by constructing a new characterization of partial recursive functions and then using the Basic Result to show the existence of a new set of instructions within the standard characterization that can compute the same function.

This new set of instructions is generated by computing the **s-n-m function** which takes as input the Godel Number of the original function and the fixed values of the first m variables and outputs the Godel Number for the partial recursive function which computes the function for all values of $z_1, z_2, \dots, z_n$ with fixed values of x and $y_1, y_2, \dots, y_m$.

The s-m-n function is itself partial recursive.

Theorem 1.10 (Composite of Partial Recursive Functions are Partial Recursive). *There is a recursive function g of 2 variables such that for all x, y -*

$$\varphi_{g(x,y)} = \varphi_x \circ \varphi_y = \lambda z [\varphi_x(\varphi_y(z))]$$

Proof. By **Church's Thesis**, for any given x, y , $f = \varphi_x \circ \varphi_y$ is a partial recursive function. This implies that there exists a Godel Number $g(x, y)$ such that $\varphi_{g(x,y)} = f$.

Define -

$$\theta(x, y, z) = \varphi_{x_0}(x, \varphi_{x_0}(y, z)) = \varphi_x(\varphi_y(z))$$

where x_0 is the Godel Number of the universal partial function Ψ . Then, θ is a partial recursive function of 3 variables and therefore has a godel number, say x_1

Then, by the **s-m-n Theorem**, we have -

$$\varphi_x \varphi_y = \lambda z [\varphi_{x_1}(x, y, z)] = \varphi_{s_1^2(x_1, x, y)}$$

where s_1^2 is the s-m-n function for $m = 2$ and $n = 1$.

$\lambda xy [s_1^2(x_1, x, y)]$ gives us $g(x, y)$. □

Note 1.4.3.2

The main idea in the proof is to use the s-m-n theorem to construct a new partial recursive function that computes the composition of two partial recursive functions with previously defined Godel Numbers. First, we construct a function of three variables that computes the composition. We then fix the first two variables and define a new function which that computes the composition of any two partial recursive functions for any input given the Godel Numbers of the two functions.

1.5 The Halting Problem

1.5.1 Recursive Unsolvability

Can we algorithmically determine whether a given partial recursive function is defined for a given input? In other words, is there a recursive function $g(x, y)$ such that it outputs 1 if $\varphi_x(y)$ is defined and 0 otherwise?

Theorem 1.11 (Recursive Unsolvability of the Halting Problem). *There is no recursive function $g(x, y)$ such that for all x, y -*

$$g(x, y) = \begin{cases} 1 & \text{if } \varphi_x(y) \text{ is defined} \\ 0 & \text{otherwise} \end{cases}$$

Proof. Let g be defined as above. Then, we define -

$$\psi(x) = \begin{cases} 1 & \text{if } g(x, x) = 0 \\ \text{divergent} & \text{if } g(x, x) = 1 \end{cases}$$

By Church's Thesis, ψ is partial recursive. Let x_0 be the Godel Number of ψ . Then, $\varphi_{x_0}(x)$ is defined if and only if $g(x_0, x_0) = 1$. But, by the definition of ψ , we have that for $g(x_0, x_0) = 1$, $\varphi_{x_0}(x)$ is divergent,

and for $g(x_0, x_0) = 0$, $\varphi_{x_0}(x)$ is defined. This leads to a contradiction. Hence, no such function g exists. $\square$

Note 1.5.1.1

This is one of the first problems that have been shown to be recursively unsolvable.

Theorem 1.12 (The Halting Problem for Total Functions). *There is no recursive function $f(x, y)$ such that for all x, y -*

$$f(x, y) = \begin{cases} 1 & \text{if } \varphi_x(y) \text{ is defined and total} \\ 0 & \text{otherwise} \end{cases}$$

Proof. We assume such a function f exists. We define a new function g as follows -

$$\begin{aligned} g(0) &= \mu y[f(y) = 1] \\ g(x+1) &= \mu y[(y > g(x)) \wedge (f(y) = 1)] \end{aligned}$$

Note 1.5.1.2

Definition 1.5.1.1

The μ Operator

The μ operator is an operator that is used to define a function that returns the smallest natural number that satisfies a given condition.

Following the above definition, we see that $g(0)$ is the smallest Godel Number for a total function, and $g(x+1)$ is the smallest Godel Number for a total function that is greater than $g(x)$. $g(x)$ essentially lists all the Godel Numbers for total functions in increasing order.

Since, $f(y) = 1$ for infinitely many y (there are infinitely many total functions), we can see that g is a total function. Now, we define a new function h as follows -

$$h(x) = \lambda x[\varphi_{g(x)}(x) + 1]$$

Now, since g is a total function which returns the Godel Number for x^{th} total function, we have $\varphi_{g(x)}$ is always a total function. Hence, h is a total function. Therefore, by Church's Thesis, h is a recursive total function. Let h_0 be the Godel Number of h . Then, we have that $g^{-1}(h_0)$ is defined and let it be denoted by y_0 . Now, we have that -

$$\varphi_{h_0}(y_0) = \varphi_{g(y_0)}(y_0) + 1 = \varphi_{h_0}(y_0) + 1$$

This leads to a contradiction, since it implies that $\varphi_{h_0}(y_0)$ is not defined (and h is a total function). Hence, no such function f exists. $\square$

1.6 Recursiveness

The property of being in the class of partial recursive functions is called recursiveness. Alternatively, we may call partial recursive functions effectively computable, recursively computable, or algorithmic.

1.6.1 Codings

Partial recursive functions are themselves mappings from $\mathbb{N}$ to $\mathbb{N}$. However, the notion of an algorithm is not restricted only to numerical functions. We can have algorithms that operate on general kinds of symbolic inputs. The approach to include these broader class of algorithms with non-numerical inputs and/or outputs is described in this subsection.

We can define a coding of a set of symbols, such as the set of all characters in the English alphabet, as

a mapping from the set of symbols to $\mathbb{N}$. For example, we can define a coding for the English alphabet as follows -

$$\begin{aligned} a &\rightarrow 1 \\ b &\rightarrow 2 \\ c &\rightarrow 3 \\ d &\rightarrow 4 \\ &\vdots \\ z &\rightarrow 26 \end{aligned}$$

In essence, we identify each symbol with a unique integer key or label. The coding must be chosen in such a way that it is reversible, i.e. we can uniquely identify each symbol from its integer key, and that it must generated itself by some sort of informal algorithmic procedure. A coding is only used when there is an informal algorithmic procedure to recognize the symbols where they are uncoded.

Note 1.6.1.1

Godel Numbers are a special case of codings, which map sets of instructions to integers. The code numbers in this case are called the ***Godel Numbers*** of the sets of instructions.

Theorem 1.13 (Correspondence of partial functions with algorithmic mapping on uncoded expressions). *Every Informal mapping defined on a set of symbols S has a corresponding formal mapping defined as a partial recursive function and vice versa.*

Proof. Let γ be the coding map from S **onto** $\mathbb{N}$, that is -

$$\gamma : S \rightarrow \mathbb{N}$$

Then γ^{-1} denotes the decoding map from $\mathbb{N}$ to S . Both γ and γ^{-1} are algorithmic. Therefore, by Church's Thesis, they are partial recursive. If φ is any partial recursive function, it is both formally and informally algorithmic. Therefore, we can define a new mapping δ as follows -

$$\delta : S \rightarrow S = \gamma^{-1}\varphi\gamma$$

We can similarly construct a formal mapping φ using the above defined coding, decoding and informal mappings. The mapping φ is said to be partial recursive through Church's Thesis. Hence proved. $\square$

1.6.2 The mu (μ) operator

We use the definition provided earlier for the μ operator.

Theorem 1.14 (The Mu Theorem). ***The Mu Theorem***
Let f be a total recursive function of $k + 1$ variables; then -

$$\Psi = \lambda x_1, x_2, \dots, x_k [\mu y [f(x_1, x_2, \dots, x_k, y) = 1]]$$

is a partial recursive function of k variables.

Proof. To compute $\Psi(x_1, x_2, \dots, x_k)$, we need to compute iteratively -

$$f(x_1, \dots, x_k, 0), f(x_1, \dots, x_k, 1), \dots$$

until we find the first y where f evaluates to 1. This is the value for Ψ . Now, since f is a total recursive function, it will terminate for all combinations of inputs. Therefore, we have determined an algorithmic approach to evaluating Ψ , and therefore by Church's Thesis, it is a partial recursive function. $\square$

Note 1.6.2.1

This works in the general case only for total recursive f . This is due to the fact that the totality of f guarantees that the subcomputations for Ψ always terminate.

Theorem 1.15 (Kleene Normal-form Theorem). ***Kleene Normal-form Theorem***

There exist fixed recursive functions p and t of 1 and 3 variables respectively such that for all z ,

$$\varphi_z = \lambda x[p(\mu y[t(z, x, y) = 1])]$$

Proof. Define the function s as follows:

$$s(z, x, y, w) = \begin{cases} 1 & \text{if } P_z \text{ with input } x \text{ yields output } y, \text{ in fewer than } w \text{ steps;} \\ 0 & \text{otherwise} \end{cases}$$

By Church's Thesis, s is recursive. Define p and q by -

$$\begin{aligned} p &= \lambda x[\text{exponent of 3 in prime decomposition of } x + 1] \\ q &= \lambda x[\text{exponent of 2 in prime decomposition of } x + 1] \end{aligned}$$

Define t by -

$$t = \lambda xyz[s(z, x, p(y), q(y))]$$

By Church's Thesis, p, q are recursive and hence t is recursive. □

Note 1.6.2.2

This theorem enables us to essentially have fixed functions which can simulate the outputs for any set of Instructions P_z . The universal function can be derived as a direct consequence of this theorem.

Theorem 1.16 (Ex. 2-8 theorem). *There is no recursive function f of two variables such that for all x and z , P_z applied to x yields an output $\iff P_z$ applied to x yields an output in fewer than $f(z, x)$ steps.*

Proof. Let us suppose such a f exists. Define h as follows -

$$h(x, z) = \begin{cases} 1 & \text{if } P_z \text{ halts in } f(z, x) \text{ steps} \\ 0 & \text{otherwise} \end{cases}$$

Since f is assumed to be recursive, the subcomputations of $h(x, z)$ will always halt and return an output. In other words, h is a recursive function by Church's Thesis. However, h simulates the halting problem, which has shown to be recursively unsolvable. This is a contradiction. Hence, no such f exists. □

With this proof, we end the chapter and shift our focus to unsolvable problems.

Chapter 2

Unsolvable Problems

2.1 Further Examples of Recursive Unsolvability

2.1.1 Methods of Proving Unsolvability

There are two different methods for demonstrating recursive unsolvability -

1. **The Direct Method**- We usually make a diagonal argument, showing that solvability would lead to contradiction.

Example 2.1.1.1 - The Halting Problem

2. **Indirect Method**- This method takes another problem, which has been shown to be unsolvable, and then show that the solvability of the above problem would lead to the solvability of the unsolvable problem.

Example 2.1.1.2 - Theorem 1.16

The second method is called the **Reduction Method**. It is often more useful and convenient than the direct method.

Theorem 2.1 (Unsolvable (a)). *There is no recursive function g such that-*

$$g(x) = \begin{cases} 1 & \text{if } \varphi_x \text{ is a constant function} \\ \text{divergent} & \text{otherwise} \end{cases}$$

Proof. Assume such a function g exists. Then, we can define a new function ψ as follows -

$$\psi(x, y) = \begin{cases} 0 & \text{if } \varphi_x(x) \text{ converges} \\ \text{divergent} & \text{otherwise} \end{cases}$$

By s-m-n theorem, we can find a function h such that -

$$\psi(x, y) = \varphi_{h(x)}(y)$$

Therefore, $\varphi_{h(x)}(x)$ is defined and constant if and only if $\varphi_x(x)$ converges. Now -

$$g(h(x)) = \begin{cases} 1 & \text{if } \varphi_{h(x)}(x) \text{ is a constant function} \\ \text{divergent} & \text{otherwise} \end{cases}$$

But, by the definition of ψ , we have that -

$$g(h(x)) = \begin{cases} 1 & \text{if } \varphi_x(x) \text{ converges} \\ \text{divergent} & \text{otherwise} \end{cases}$$

This leads to a contradiction, since it implies that g can be used to determine whether $\varphi_x(x)$ converges or not, which is the Halting Problem. Hence, no such function g exists. $\square$

Note 2.1.1.1

The basic idea in the proof is to construct a function that can take the definition of the function in the theorem and then manipulate it to show that it can be used to solve the Halting Problem. We can do this by constructing a new function and then applying the s-m-n theorem, or use the enumeration theorem to find a universal function that can similarly demonstrate the unsolvability of the problem.

I do not provide the proof for the other theorems in this section, since they follow similar patterns of proof. All other proofs can be found in the book.

2.2 Existence of certain partial recursive functions

2.2.1 Extension of Partial Recursive Functions

Can we extend a partial recursive function to a total recursive function for all partial recursive functions? In other words, can we always find a total recursive function that is an extension of a given partial recursive function?

Theorem 2.2 (Extension of Partial Recursive Functions). *There exists a partial recursive function φ such that there does not exist a total recursive function f such that for all x , if φ converges $\varphi(x) = f(x)$. In logical notation -*

$$\exists \varphi [\nexists f [\forall x [\varphi(x) \text{ converges} \implies \varphi(x) = f(x)]]]$$

Proof. Assume such a function f exists. Then, we can define a new function ψ as follows -

$$\psi(x) = \begin{cases} \varphi_x(x) + 1 & \text{if } \varphi_x(x) \text{ converges} \\ \text{divergent} & \text{otherwise} \end{cases}$$

Then since f is a recursive function, it can be notated as φ_y for some y . Now, f is total, and hence φ_y is convergent for all x . Now, we have -

$$\psi(y) = \varphi_y(y) + 1 = f(y) + 1$$

Hence, no such function f can exist. □

Can each partial recursive function be obtained by a single application of the μ operator on a recursive function? In other words, can we always find a recursive function f such that for all x , $\varphi_x = \mu y [f(x, y) = 1]$?

Theorem 2.3 (Existence of Partial Recursive Functions not obtained by μ). *There exists a partial recursive function φ such that there does not exist a recursive function f such that for all x, y , $\varphi_x = \mu y [f(x, y) = 1]$. In logical notation -*

$$\exists \varphi [\nexists f [\forall x, y [\varphi_x = \lambda x [\mu y [f(x, y) = 1]]]]]$$

Proof. Assume such a function exists. We define a new function ψ as follows -

$$\psi(x) = \begin{cases} x & \text{if } \varphi_x(x) \text{ converges} \\ \text{divergent} & \text{otherwise} \end{cases}$$

Then, we have that if $\varphi_x(x)$ converges, $f(x, x) = 1$, and $f(x, x) \neq 1$ otherwise. This is a contradiction, since it implies that f can be used to determine whether $\varphi_x(x)$ converges or not, which is the Halting Problem. Hence, no such function f exists. □

Theorem 2.4 (Important Corollary of the Halting Problem). *There does not exist a recursive function f such that for all x -*

$$f(x) = \begin{cases} 1 & \text{if } \varphi_{h(x)}(h(x)) \downarrow \\ 0 & \text{otherwise} \end{cases}$$

where h is any recursive function.

Proof. Assume such a function f exists. We define a new function F as follows -

$$F(z, y, w) = \varphi_z(y)$$

Then by the s-m-n theorem, we can find a recursive function p such that -

$$\varphi_{p(z,y)}(w) = \varphi_z(y)$$

We can then define a new function g as follows -

$$g(z, y) = \mu x(h(x) = p(z, y))$$

By the definition of f , we have that -

$$f(h(g((z, y)))) = f(p(z, y)) = \begin{cases} 1 & \text{if } \varphi_{p(z,y)}(p(z, y)) \downarrow \\ 0 & \text{otherwise} \end{cases}$$

But, by the definition of $p(z, y)$, we have -

$$f(p(z, y)) = \begin{cases} 1 & \text{if } \varphi_z(y) \downarrow \\ 0 & \text{otherwise} \end{cases}$$

This leads to a contradiction, since it implies that f can be used to determine whether $\varphi_z(y)$ converges or not, which is the Halting Problem. Hence, no such function f exists. $\square$

2.3 Exercises

2.3.1 Exercises from Chapter 1

Exercise 2.1:

We have -

$$g(x) = \begin{cases} 1 & \text{if a consecutive run of at least } x \text{ 5's occurs in the decimal expansion of } \pi \\ 0 & \text{otherwise} \end{cases}$$

We can see that either g is a constant function or, for $g(x) = 1$ for $x \leq k$ for some k and $g(x) = 0$ for $x > k$.

In the first case, g can be trivially said to be primitive recursive, since it is a constant function.

For the second case, we define functions h_1 and f as follows -

$$\begin{aligned} h_1 &= \lambda x[\max(0, x - 1)] \\ f &= \lambda x[\min(x, 1)] \end{aligned}$$

We can provide a primitive recursive derivation for h_1 as follows -

$$\begin{aligned} h_1(0) &= 0 \\ h_1(x + 1) &= x \end{aligned}$$

Similarly, we can provide a primitive recursive derivation for f as follows -

$$\begin{aligned} f(0) &= 0 \\ f(x + 1) &= 1 \end{aligned}$$

We can now define a function h as follows -

$$h = \lambda xy[\max(0, x - y)]$$

A primitive recursive definition for h is as follows -

$$\begin{aligned} h(x, 0) &= 0 \\ h(x, y + 1) &= h_1(h(x, y)) \end{aligned}$$

Note 2.3.1.1

$h(x, y + 1)$ is defined as $h_1(h(x, y))$, which is the same as $\max(0, h(x, y) - 1)$. Let us evaluate $h(3, 5)$. -

$$\begin{aligned}
 h(3, 5) &= h_1(h(3, 4)) \\
 h(3, 4) &= h_1(h(3, 3)) \\
 h(3, 3) &= h_1(h(3, 2)) \\
 h(3, 2) &= h_1(h(3, 1)) \\
 h(3, 1) &= h_1(h(3, 0)) \\
 h(3, 0) &= 3 \\
 h(3, 1) &= h_1(3) = 2 \\
 h(3, 2) &= h_1(2) = 1 \\
 h(3, 3) &= h_1(1) = 0 \\
 h(3, 4) &= h_1(0) = 0 \\
 h(3, 5) &= h_1(0) = 0
 \end{aligned}$$

Hence, $h(3, 5) = 0$. Similarly, we can evaluate $h(3, 2)$ as follows -

$$\begin{aligned}
 h(3, 2) &= h_1(h(3, 1)) \\
 h(3, 1) &= h_1(h(3, 0)) \\
 h(3, 0) &= 3 \\
 h(3, 1) &= h_1(3) = 2 \\
 h(3, 2) &= h_1(2) = 1
 \end{aligned}$$

Hence, $h(3, 2) = 1$, which is the expected output.

h works by subtracting 1 from x y times and returning the result. If the result is less than 0, it returns 0. This is the same as $\max(0, x - y)$.

Now we can give a derivation for g as follows-

$$g(x) = \lambda x[f(h(k, x))]$$

Here, k is the fixed number that is the maximum number of 5's that can occur simultaneously in the decimal expansion of π .

Note 2.3.1.2

We can see that g is primitive recursive, since it is a composition of primitive recursive functions. If $x < k$ then $h(k, x) = \max(0, k - x) = k - x$, and hence $f(h(k, x)) = f(k - x) = 1$. If $x \geq k$, then $h(k, x) = 0$ and hence $f(h(k, x)) = f(0) = 0$. Hence, g is primitive recursive.

Exercise 2.2:

We have -

$$f(x) = \begin{cases} 1 & \text{if } \varphi_x(x) = 1 \\ 0 & \text{otherwise} \end{cases}$$

We assume such a function f exists. Then, we can define a new function Ψ as follows -

$$\Psi(x, y) = \begin{cases} 1 & \text{if } \varphi_x(x) \text{ converges} \\ \text{divergent} & \text{otherwise} \end{cases}$$

By the s-m-n theorem, we can find a recursive function h such that -

$$\Psi(x, y) = \varphi_{h(x)}(y)$$

Therefore, $\varphi_{h(x)}(x)$ is defined and converges if and only if $\varphi_x(x)$ converges. Now,

$$f(h(x)) = \begin{cases} 1 & \text{if } \varphi_{h(x)}(h(x)) = 1 \\ 0 & \text{otherwise} \end{cases}$$

But, by the definition of Ψ , we have that -

$$f(h(x)) = \begin{cases} 1 & \text{if } \varphi_{h(x)}(h(x)) \text{ converges} \\ 0 & \text{otherwise} \end{cases}$$

This leads to a contradiction, since it implies that f can be used to determine whether $\varphi_x(x)$ converges or not (due to the totality of h), which is the Halting Problem. Hence, no such function f exists.

Exercise 2.3:

We have a list of symbolic derivations of primitive recursive functions. Let f_x be the function determined by the $(x + 1)$ st derivation in the list. We can define a new function g as follows -

$$g = \lambda xy[f_x(y)]$$

We can see that g is recursive, since $f_x(y)$ is recursive and defined for all x and y by definition. However, g is not primitive recursive. If it was primitive recursive, then it would be the universal primitive recursive function. We could then define a new function h as follows -

$$h(x) = g(x, x) + 1$$

Then, h would be a primitive recursive function that is not in the list of primitive recursive functions, which is a contradiction. Hence, g is not primitive recursive. This is the diagonalization argument, which is a common technique used to prove the existence of functions that are not in a given list of functions.

Exercise 2.4:

We define a function f as follows -

$$f(x, y_1, y_2, y_3, \dots, y_k) = \begin{cases} 1 & \text{if } \varphi_x(y_1, y_2, y_3, \dots, y_k) \text{ converges} \\ \text{divergent} & \text{otherwise} \end{cases}$$

By s-m-n theorem, we can find a recursive function h such that -

$$f(x, y_1, y_2, y_3, \dots, y_k) = \varphi_{h(y_1, y_2, \dots, y_k)}(x)$$

Define a new function g as follows -

$$g(x) = \begin{cases} 1 & \text{if } f(h(x, x, x, \dots, x), x, \dots, x) \text{ converges} \\ \text{divergent} & \text{otherwise} \end{cases}$$

Part II

Data Structures and Algorithms - Analysis and Design

Chapter 3

Basic Terminology and Definitions

3.1 Foundational Concepts

3.1.1 Introduction

We use algorithms to solve problems in a *deterministic manner*. An algorithm, informally, is a set of instructions that can be followed, with a finite number of steps, and with a way to note intermediate results and progress within the set of instructions, to arrive at a solution given a set of inputs.

Many applications of algorithms being used to solve problems can be found in the real world, such as *sorting a list of products by price* on an e-commerce website, *finding the shortest path* between two points on a map, or *searching for a specific item* in a database.

There are many other important applications of algorithms, such as in cryptography, data compression, and machine learning. Algorithms are also used in many other fields, such as physics, chemistry, biology, and economics. All of this means that algorithms are a fundamental part of computer science and are essential for solving many real-world problems. Hence, it is important to understand the basic concepts and definitions related to algorithms.

We begin by defining some basic concepts and terms related to algorithms, data structures, and efficiency. These definitions will be used throughout the notes to provide a common understanding of the concepts and terms used in the field of algorithms and data structures.

Note 3.1.1.1

For a more formal definition and study of algorithms, refer to Part I of the notes, where algorithms are related to recursive functions and given formal characterizations.

3.1.2 Definitions

Definition 3.1.2.1

Data Structure

A data structure is a way of storing and organizing data to facilitate efficient access and modification. Each data structure is designed to organize data in a specific way to suit a particular purpose. As such, it is important to choose the right data structure for a given problem to find the most efficient solution.

Definition 3.1.2.2

Hard Problems and Efficiency

We usually use the speed taken for the execution of an algorithm as a measure of its efficiency. In other cases, we may use the amount of memory or the number of operations performed as a measure of efficiency. Calculating the efficiency of an algorithm is fundamental to design and understand algorithms as a whole, and is a key part of the study of algorithms.

A hard problem is a problem that can not be solved in a *reasonable amount of time* or with a

reasonable amount of resources. The definition of a reasonable amount of time or resources will not be defined here. Complexity theory is the study of the efficiency of algorithms and the classification of problems based on their hardness. The most interesting problems in complexity theory are those that are *NP-hard* or *NP-complete*. These problems are considered to be the hardest problems in complexity theory, and no algorithm is known to solve them essentially

Chapter 4

Basic Concepts of Algorithm Analysis and Design

4.1 Insertion Sort

4.1.1 The Sorting Problem

The sorting problem is a fundamental problem in computer science, and it is the problem of arranging a set of elements in a specific order. The most common order is the ***ascending order***, where the elements are arranged from the smallest to the largest. However, other orders, such as descending order or alphabetical order, can also be used.

Part III

Extra Reading - Recursion and Incompleteness Theorems

Chapter 5

Introduction and Definitions (Incompleteness)

5.1 Statement of the Problem and Prerequisite Definitions

We deal with the Incompleteness Theorem, as stated below.

Theorem 5.1 (The Incompleteness Theorem). ***The Incompleteness Theorem***
Informally, under certain conditions, there exist true but unprovable statements in any language.

5.1.1 Definitions

Definition 5.1.1.1

Language

An informal definition would be that, a language is completely defined on the basis of the alphabet L and the subset T of True Statements in L^∞ . $\langle L, T \rangle$ is called the fundamental pair.

Definition 5.1.1.2

Alphabets and Words

A finite list of elementary signs that can be used in combinations as a sequence. Each sign is known as a letter. A finite sequence of letters is called a word.

If L is an alphabet, L^∞ is the set of all words that can be formed from the letters in the alphabet. Any sentence or test constructed by expanding the alphabet to include punctuation, spaces etc. can be considered a word.

Any set contained in L^∞ is called a word set in the alphabet L .

Definition 5.1.1.3

Set of True Statements

The subset T of L^∞ is the set of all true statements in the language L . We assume that T is given to us, and we skip the problem of specifying the criteria for a statement to be true or false.

Definition 5.1.1.4

Proof

A proof is informally an argument that we make to make sure that a statement is true or false. Formally, a proof can be defined as a word in some alphabet, say P . Then all proofs are words in P^∞ .

We allow variation of P and different proof-subsets for different concepts of proofs. We also

require there be an algorithmic procedure (as defined formally in the first part of the notes) to determine whether a word is a proof or not.

We also assume that there exists an algorithmic procedure to determine which statement is being proven given a proof.

Definition 5.1.1.5

Unprovable Statements

A statement is unprovable if we can not prove it to be true or false, i.e. it does not have a proof.

5.2 Requirements for a deductive system

Therefore, we have a language alphabet L , a set of true statements T in L^∞ , and a set of proofs P in $\mathbf{P}^\infty$.

We also have a function δ that takes a proof p and returns the statement that is being proven by the proof p . We assume that δ is a recursive function, i.e. it can be computed by an algorithm.

Every proof p in P is a proof of the word $\delta(p)$ in L^∞ .

Note 5.2.0.1

Necessarily, the domain of δ , Δ satisfies $P \subseteq \Delta \subseteq \mathbf{P}^\infty$. The range of δ is L^∞ .

Then the function can be defined as -

$$\delta : \Delta \rightarrow L^\infty$$

We say a system is a ***deductive system*** if it satisfies all the above requirements. In formal notation, a triple $\langle \mathbf{P}, P, \delta \rangle$ is a deductive system over L if it satisfies the above requirements.